

Relatório Técnico: Processamento Paralelo de Imagens

Projeto de Solução Distribuída, Etapa 1 070080 Sistemas Distribuídos e Paralelos, Prof. Fábio Rocha de Araújo, turma CC6MA

#	Integrante	Matrícula
1	Benjamin Yuji Suzuki	24070067
2	Ryan Iketani Cavalcanti	24070065
3	Felipe de Freitas da Silva	24070063
4	Lucas Coelho	24070055

Repositório: github.com/NotRyanIKC/Sistema-Distribuidos-et1

1. O problema

Uma linha de inspeção fotografa peças em uma esteira e precisa classificar cada foto como **defeituosa** ou **conforme**. Cada imagem Full HD (1920x1080) passa pelo mesmo pipeline:

1. escala de cinza (ITU-R BT.601);
2. suavização gaussiana com núcleo 5x5 (convolução);
3. gradiente de Sobel horizontal e vertical e magnitude;
4. limiarização de Otsu, gerando uma imagem binária de bordas;
5. razão de pixels de borda; acima de 0,5% a peça é classificada como defeituosa.

Entrada: 1.200 imagens PNG sem perdas geradas por `gerar_dataset.py` a partir de sementes fixas, cerca de 3,2 GB. O dataset é determinístico: qualquer máquina gera os mesmos bytes. Cerca de metade das imagens recebe manchas que simulam defeitos.

Unidade de trabalho: uma imagem. O resultado de uma imagem não depende de nenhuma outra, então o problema é embaraçosamente paralelo.

Volume: a versão sequencial leva em média 128,7 s (2 min 9 s) na instância, cerca de 0,107 s por imagem.

Verificação do resultado: para cada imagem é calculado o SHA-256 da imagem binária de bordas. A versão paralela precisa produzir o mesmo hash em todas as 1.200 imagens, as mesmas classificações, os mesmos totais e o mesmo **hash consolidado** do lote (SHA-256 da lista ordenada de `arquivo|hash|classificação`). `verificar_consistencia.py` faz essa comparação e o `benchmark.py` a executa após cada par de medições.

2. Estratégia de paralelização

Decisão	Escolha	Justificativa
Dados ou tarefas	Paralelismo de dados	Todas as imagens passam pelo mesmo pipeline; o que se divide é o conjunto de imagens.
Processo ou thread	Processos (<code>multiprocessing.Pool</code>)	O trabalho é limitado por processador: convoluções e operações aritméticas sobre cerca de 2 milhões de pixels por imagem, com o disco já em cache. No CPython o GIL permite que só uma thread execute bytecode por vez, então threads dariam speedup próximo de 1. Cada processo tem seu próprio interpretador e seu próprio GIL.
Número de trabalhadores	2 processos	Um por núcleo físico da instância (m7a.large, 2 vCPUs, cada vCPU um núcleo). Mais processos que núcleos só acrescentariam troca de contexto. A ficha previa 4 processos em uma c6i.xlarge, mas o AWS Academy Learner Lab só permite instâncias até o tamanho large (2 vCPUs).
Divisão do trabalho	Fila dinâmica (<code>imap_unordered, chunksize=2</code>)	Cada processo pega as próximas 2 imagens assim que termina as anteriores. Uma divisão estática em 2 blocos fixos deixaria processos ociosos se um bloco demorasse mais. Com lotes pequenos, o desequilíbrio no fim da fila fica em no máximo 2 imagens.

Os workers recebem apenas o caminho do arquivo e leem a imagem por conta própria. Voltam ao processo principal só o dicionário de métricas de cada imagem, algumas centenas de bytes, o que mantém baixo o custo de comunicação entre processos.

3. Seção crítica e primitiva

Estado compartilhado: um vetor de três inteiros em memória compartilhada (`multiprocessing.RawArray`) com os totais de imagens processadas, defeituosas e conformes, escrito pelos 2 processos, e um acumulador do tempo de espera pela trava (`RawValue`), usado na análise.

`RawArray` não tem trava embutida. O incremento `contadores[i] += 1` são três passos: ler o valor, somar 1 e

escrever. Se dois processos lerem o mesmo valor antes de qualquer um escrever, um incremento se perde.

Seção crítica (paralelo.py, função agregar):

```
def agregar(defeituosa: bool) -> None:
    t0 = time.perf_counter()
    with _lock:
        # entra na seção crítica
        _espera_lock.value += time.perf_counter() - t0
        _contadores[PROCESSADAS] += 1
        if defeituosa:
            _contadores[DEFEITUOSAS] += 1
        else:
            _contadores[CONFORMES] += 1
    # sai da seção crítica
```

Primitiva: multiprocessing.Lock, um mutex entre processos, criado no processo principal e entregue a cada worker pelo initializer do pool. A trava envolve só a atualização dos contadores. A leitura da imagem e todo o pipeline (cerca de 99% do tempo de cada tarefa) ficam fora dela. Uma trava em volta do laço inteiro também seria correta, mas serializaria o trabalho e o speedup cairia para perto de 1.

Prova de que o resultado é estável:

- demo_secao_critica.py: 4 processos fazem 200 mil incrementos cada no mesmo RawValue. Sem a trava o total fica muito abaixo de 800.000 (incrementos perdidos); com a trava o total é sempre 800.000.
- testar_race_condition.py: roda a versão paralela 5 vezes sobre a mesma amostra e confere, em cada rodada, que o contador compartilhado é igual ao número de imagens e que totais e hash consolidado são idênticos.

Rodada	Contador	Defeituosas	Conformes	Hash consolidado
1	100	37	63	e48125f9959fa846...
2	100	37	63	e48125f9959fa846...
3	100	37	63	e48125f9959fa846...
4	100	37	63	e48125f9959fa846...
5	100	37	63	e48125f9959fa846...

Amostra fixa de 100 imagens. Nas 5 rodadas o contador chegou a 100, os totais e o hash consolidado foram idênticos, e a espera total pela trava ficou entre 0,7 e 0,8 ms por rodada.

Relógios lógicos: a solução usa memória compartilhada em uma única máquina, sem troca de mensagens entre nós. A ordem das atualizações é garantida pelo lock e o término de todos os workers pelo join do pool, então relógios de Lamport ou vetoriais não se aplicam nesta etapa.

4. Recursos provisionados

Criados pela equipe no console do AWS Academy Learner Lab (o repositório também traz provisionar_aws.sh, que cria os mesmos recursos pela AWS CLI).

Mudança em relação à ficha: a ficha registrou uma c6i.xlarge (4 vCPUs). O Learner Lab só permite instâncias até o tamanho large, com 2 vCPUs. Entre as opções de 2 vCPUs a equipe escolheu a m7a.large porque, nela, cada vCPU é um núcleo físico; em uma c6i.large ou t3.large os 2 vCPUs são duas threads do mesmo núcleo, e o paralelismo real seria praticamente nulo. Os 8 GiB de memória comportam o dataset inteiro (3,2 GB) no cache de página.

Item	Valor
Região	us-east-1 (Norte da Virgínia)
Zonas de disponibilidade	1 (us-east-1a)
SLA aplicável	99,5% para instância isolada (até 216 min de indisponibilidade em 30 dias). É um processamento em lote para medição; duas zonas (99,99%) não trariam ganho ao experimento e dobrariam o custo.
Instância	1x m7a.large, ID i-058ef46df189265c5
CPU	2 vCPUs AMD EPYC de 4ª geração; na família m7a cada vCPU é um núcleo físico (sem SMT)
Memória	8 GiB
Sistema	Amazon Linux 2023 (x86_64), Python 3.11 em ambiente virtual
Armazenamento	EBS gp3 50 GB, 3.000 IOPS, 125 MB/s. Entrada em ~/benchmark/imagens (3,2 GB), saídas em ~/benchmark/resultados
Ciclo de vida	Criada para as medições e a apresentação, parada ao fim de cada dia (o próprio lab também a para ao encerrar a sessão, e ela volta com outro IP público) e encerrada após a entrega.

Grupo de segurança sg-benchmark-etapa1:

Porta	O que fica nela	Origem	Motivo
22/tcp	SSH (administração)	18.206.107.24/29 (faixa do EC2 Instance Connect em us-east-1)	A equipe administra pelo terminal do console da AWS. O SSH só aceita conexões vindas do serviço Instance Connect, que por sua vez só atende quem está autenticado na conta do lab.
8000/tcp	servidor_status.py, página somente leitura com os resultados	IP público da equipe, /32 (regra My IP)	Porta do serviço. Não executa comandos nem aceita envio de dados.

Nenhuma outra porta de entrada está aberta. A porta 22 não está aberta para 0.0.0.0/0 porque SSH exposto à internet inteira recebe varredura automatizada e tentativas de força bruta em minutos; restringir a origem à faixa /29 do EC2 Instance Connect faz com que só conexões intermediadas pela AWS, e portanto autenticadas na conta do lab, cheguem à porta. A porta do serviço fica limitada ao IP da equipe.

5. Medição de desempenho

Método: benchmark.py roda na própria instância. Antes das medições, lê todas as imagens uma vez para que estejam no cache de página (3,2 GB cabem em 8 GiB), assim nenhuma das versões paga sozinha a leitura do EBS. Depois executa, 3 vezes e alternadamente, a versão sequencial e a paralela com a mesma entrada, e mede o tempo de parede de cada execução com time.perf_counter (inclui iniciar o interpretador, que as duas versões pagam igualmente). Cada par de execuções passa pelo verificador de consistência.

Instância m7a.large (2 vCPUs), 2 processos, 1.200 imagens, 3 repetições de cada versão, mesma entrada.

Repetição	Sequencial (s)	Paralelo (s)	Speedup
1	127,82	77,01	1,66x
2	129,64	80,58	1,61x
3	128,58	73,13	1,76x
Média	128,68	76,91	1,67x

Desvio padrão: sequencial 0,91 s, paralelo 3,73 s. Todas as repetições passaram no verificador de consistência (1.200 hashes idênticos).

Fração paralelizável: a ficha estimou $p = 0,95$. O benchmark mede p na própria execução sequencial, como o tempo gasto dentro de processar_imagem dividido pelo tempo total: **$p = 0,9877$** . O restante (1,2%) é iniciar o interpretador, listar os arquivos e gravar o JSON e o CSV.

Lei de Amdahl, com $n = 2$:

- com p da ficha: $S = 1 / (0,05 + 0,95/2) = \mathbf{1,90x}$ (a ficha calculou 3,48x para $n = 4$);
- com p medido: $S = 1 / (0,0123 + 0,9877/2) = \mathbf{1,98x}$.

Speedup medido: 1,67x, eficiência de 84% (speedup / 2), 16% abaixo do teto de Amdahl com p medido.

Escalabilidade:

Processos	Tempo (s)	Speedup	Amdahl
1	130,08	0,99x	1,00x
2	78,89	1,63x	1,98x

Com 1 processo, a versão paralela leva só 1% a mais que a sequencial: esse é o custo do pool e da comunicação

entre processos, isolado do paralelismo.

6. O que limitou o ganho

1. **Contenção entre os núcleos (principal fator).** Os dois núcleos dividem o cache L3 e o barramento de memória, e cada convolução percorre matrizes de cerca de 8 MB, maiores que os caches privados. O benchmark mede isso diretamente: a soma dos tempos por imagem na versão paralela foi de 151,7 s contra 127,1 s na sequencial, ou seja, cada imagem ficou **1,19x mais lenta** com os 2 processos rodando juntos. Dividindo o teto de Amdahl por esse fator, $1,98 / 1,19 = 1,66x$, praticamente o speedup medido (1,67x). A contenção explica quase toda a diferença.
2. **Parte sequencial.** Iniciar o interpretador, listar os arquivos, ordenar e gravar o JSON e o CSV não se paraleliza; é o $(1 - p) = 1,2\%$ da lei de Amdahl, já contabilizado no teto de 1,98x.
3. **Comunicação e coordenação.** Iniciar os processos, enviar os caminhos, receber os dicionários de resultado e o desequilíbrio no fim da fila somaram 2,1 s, 1,4% do tempo disponível dos processos. A variação maior do tempo paralelo (desvio de 3,73 s contra 0,91 s) vem do escalonamento dos dois processos pelo sistema operacional.
4. **Espera na seção crítica.** Os workers esperaram 8,5 ms no total para entrar no lock, 0,006% do tempo. Como a seção crítica tem só três incrementos, a trava não limita o ganho.

Conclusão: a versão paralela produz exatamente o mesmo resultado que a sequencial (1.200 hashes idênticos e mesmo hash consolidado), a seção crítica protegida por `multiprocessing.Lock` não perde incrementos em nenhuma rodada, e o speedup de 1,67x fica 16% abaixo do teto de Amdahl (1,98x) quase inteiramente por contenção de cache e memória entre os dois núcleos, e não por causa da sincronização. O limite do ganho, nesta etapa, é o número de núcleos que o Learner Lab permite por instância; distribuir o lote entre várias instâncias (o lab permite até 9) é o caminho natural para as próximas etapas.